

Database Management System

Client-Server Architecture

C++ Implementation with OOP Principles

Academia Tehnica Militara

Facultatea de Calculatoare si Informatica

Autori:

Pandele Catalin

Florea Eduard

January 2026

Contents

1	Introduction	3
1.1	Project Scope	3
1.2	Main Objectives	3
1.3	Technologies Used	3
2	System Architecture	4
2.1	General Overview	4
2.2	Main Components	4
2.2.1	Layer 1: Data Types	4
2.2.2	Layer 2: Database Structures	4
2.2.3	Layer 3: Networking & Communication	5
2.2.4	Layer 4: Business Logic	5
3	Design Patterns Used	5
3.1	Singleton Pattern	5
3.2	Factory Pattern	5
3.3	Command Pattern	6
4	Detailed Class Structure	6
4.1	DataType Class	6
4.2	IntType Class	6
4.3	VarcharType Class	8
4.4	DateType Class	9
4.5	Column Class	10
4.6	Row Class	11
4.7	Table Class	12
4.8	Database Class	14
5	Client-Server Communication	16
5.1	JSON Protocol	16
5.1.1	Client to Server Message Format	16
5.1.2	Server to Client Response Format	17
5.2	Server Class	17
5.3	ClientHandler Class	19
5.4	DatabaseManager Class	20
6	Client Implementation	21
7	Build System - CMake	23
7.1	Build and Run	24
8	Advanced Features	24
8.1	Thread Safety	24
8.2	Data Persistence	25
8.3	Error Handling	25

9	Testing and Examples	25
9.1	Complete Usage Example	25
9.2	Test Scenarios	25
10	Possible Improvements	26
10.1	Future Functionalities	26
10.2	Performance Optimizations	27
11	Conclusions	27
11.1	Accomplished Objectives	27
11.2	Lessons Learned	27
11.3	Applicability	28

Abstract

Acest document prezinta implementarea unui sistem de gestiune a bazelor de date de tip client-server, dezvoltat in C++ folosind principiile programarii orientate pe obiecte (OOP). Sistemul permite conexiuni simultane de la mai multi clienti, suportand operatii CRUD (Create, Read, Update, Delete) asupra tabelelor de date. Comunicarea intre client si server se realizeaza prin protocol JSON, iar sistemul implementeaza mecanisme de sincronizare thread-safe pentru gestionarea accesului concurent la baza de date.

1 Introduction

1.1 Project Scope

The project aims to develop a complete database management system that demonstrates knowledge in system programming, networking, and design patterns. The system implements functionalities similar to a real DBMS, but in a simplified and educational form.

1.2 Main Objectives

- Implementation of a robust client-server architecture
- Support for simultaneous connections through multi-threading
- Complete CRUD operations on data
- Data persistence through disk storage
- Communication via standardized JSON protocol
- Thread-safety for concurrent operations

1.3 Technologies Used

- **Language:** C++11/14
- **Platform:** Linux (developed on virtual machine)
- **IDE:** Visual Studio Code
- **Build System:** CMake
- **Libraries:** nlohmann/json, STL (Standard Template Library)
- **Networking:** BSD Sockets (POSIX)
- **Threading:** C++11 `std::thread` and `std::mutex`

2 System Architecture

2.1 General Overview

The system is structured in multiple layers, each with well-defined responsibilities according to SOLID principles. The architecture follows a layered model, separating business logic, communication, and data storage.

Architecture Layers:

1. **Layer 1: Data Types** - INT, VARCHAR(N), DATE
2. **Layer 2: Database Structures** - Column, Row, Table, Database
3. **Layer 3: Networking & Communication** - Server, ClientHandler, Protocol
4. **Layer 4: Business Logic** - DatabaseManager
5. **Layer 5: Application** - Main entry points

2.2 Main Components

2.2.1 Layer 1: Data Types

The lowest layer, responsible for defining and manipulating supported data types.

Classes:

- **DataType** - Abstract base class
- **IntType** - Integer type
- **VarcharType** - String with maximum length N
- **DateType** - Calendar date

Responsibilities:

- Value validation
- JSON $\leftrightarrow$ internal type conversion
- Comparison and sorting
- Serialization/deserialization

2.2.2 Layer 2: Database Structures

Data structure layer that builds fundamental database concepts.

Classes:

- **Column** - A column (name + DataType)
- **Row** - A data row
- **Table** - Complete table with CRUD operations
- **Database** - Collection of tables

2.2.3 Layer 3: Networking & Communication

Communication layer between client and server.

Classes:

- **Server** - Socket server, accepts connections
- **ClientHandler** - Manages one client on separate thread
- **Protocol** - Parse/serialize JSON messages

2.2.4 Layer 4: Business Logic

Central layer that coordinates operations and ensures thread-safety.

Classes:

- **DatabaseManager** - Thread-safe interface to Database

3 Design Patterns Used

3.1 Singleton Pattern

DatabaseManager is implemented as a singleton to ensure a single instance of the database in memory.

```
1 class DatabaseManager {
2 private:
3     static DatabaseManager* instance;
4     Database database;
5     std::mutex dbMutex;
6
7     // Private constructor
8     DatabaseManager() : database("MainDB") {}
9
10 public:
11     static DatabaseManager* getInstance() {
12         if (instance == nullptr) {
13             instance = new DatabaseManager();
14         }
15         return instance;
16     }
17
18     // Delete copy constructor and assignment
19     DatabaseManager(const DatabaseManager&) = delete;
20     DatabaseManager& operator=(const DatabaseManager&) = delete;
21 };
```

Listing 1: Singleton Implementation for DatabaseManager

3.2 Factory Pattern

For creating DataType objects, the Factory pattern is used.

```

1 class DataTypeFactory {
2 public:
3     static std::shared_ptr<DataType> createType(
4         const std::string& typeName) {
5
6         if (typeName == "INT") {
7             return std::make_shared<IntType>();
8         } else if (typeName.find("VARCHAR") == 0) {
9             int size = extractSize(typeName);
10            return std::make_shared<VarcharType>(size);
11        } else if (typeName == "DATE") {
12            return std::make_shared<DateType>();
13        }
14        return nullptr;
15    }
16 };

```

Listing 2: Factory for DataType

3.3 Command Pattern

Operations on the database (INSERT, UPDATE, DELETE) are implemented as commands.

4 Detailed Class Structure

4.1 DataType Class

```

1 class DataType {
2 protected:
3     std::string typeName;
4
5 public:
6     virtual ~DataType() = default;
7
8     // Pure virtual methods
9     virtual bool validate(const std::string& value) = 0;
10    virtual std::string toString() const = 0;
11    virtual nlohmann::json toJSON() const = 0;
12    virtual void fromJSON(const nlohmann::json& j) = 0;
13    virtual std::shared_ptr<DataType> clone() const = 0;
14
15    // Comparison
16    virtual bool equals(const DataType& other) const = 0;
17    virtual bool lessThan(const DataType& other) const = 0;
18
19    std::string getTypeName() const { return typeName; }
20 };

```

Listing 3: DataType - Abstract Base Class

4.2 IntType Class

```
1 class IntType : public DataType {
2 private:
3     int value;
4
5 public:
6     IntType() : value(0) {
7         typeName = "INT";
8     }
9
10    IntType(int val) : value(val) {
11        typeName = "INT";
12    }
13
14    bool validate(const std::string& val) override {
15        try {
16            std::stoi(val);
17            return true;
18        } catch (...) {
19            return false;
20        }
21    }
22
23    std::string toString() const override {
24        return std::to_string(value);
25    }
26
27    nlohmann::json toJSON() const override {
28        return nlohmann::json{
29            {"type", "INT"},
30            {"value", value}
31        };
32    }
33
34    void fromJSON(const nlohmann::json& j) override {
35        value = j["value"].get<int>();
36    }
37
38    std::shared_ptr<DataType> clone() const override {
39        return std::make_shared<IntType>(value);
40    }
41
42    bool equals(const DataType& other) const override {
43        if (other.getTypeName() != "INT") return false;
44        const IntType* otherInt =
45            dynamic_cast<const IntType*>(&other);
46        return value == otherInt->value;
47    }
48
49    bool lessThan(const DataType& other) const override {
50        const IntType* otherInt =
51            dynamic_cast<const IntType*>(&other);
52        return value < otherInt->value;
53    }
54
55    int getValue() const { return value; }
56    void setValue(int val) { value = val; }
```

57 };

Listing 4: IntType - Integer Type Implementation

4.3 VarcharType Class

```

1 class VarcharType : public DataType {
2 private:
3     std::string value;
4     size_t maxLength;
5
6 public:
7     VarcharType(size_t maxLen = 255)
8         : maxLength(maxLen), value("") {
9         typeName = "VARCHAR(" +
10             std::to_string(maxLength) + ")";
11     }
12
13     bool validate(const std::string& val) override {
14         return val.length() <= maxLength;
15     }
16
17     std::string toString() const override {
18         return value;
19     }
20
21     nlohmann::json toJSON() const override {
22         return nlohmann::json{
23             {"type", typeName},
24             {"value", value},
25             {"maxLength", maxLength}
26         };
27     }
28
29     void fromJSON(const nlohmann::json& j) override {
30         value = j["value"].get<std::string>();
31         maxLength = j["maxLength"].get<size_t>();
32     }
33
34     std::shared_ptr<DataType> clone() const override {
35         auto clone = std::make_shared<VarcharType>(maxLength);
36         clone->setValue(value);
37         return clone;
38     }
39
40     std::string getValue() const { return value; }
41     void setValue(const std::string& val) {
42         if (validate(val)) {
43             value = val;
44         } else {
45             throw std::invalid_argument(
46                 "String exceeds maximum length");
47         }
48     }
49 };

```

Listing 5: VarcharType - String Type with Limit

4.4 DateType Class

```

1 class DateType : public DataType {
2 private:
3     int year;
4     int month;
5     int day;
6
7 public:
8     DateType() : year(2026), month(1), day(1) {
9         typeName = "DATE";
10    }
11
12    DateType(int y, int m, int d)
13        : year(y), month(m), day(d) {
14        typeName = "DATE";
15        if (!isValid()) {
16            throw std::invalid_argument("Invalid date");
17        }
18    }
19
20    bool validate(const std::string& val) override {
21        // Format: YYYY-MM-DD
22        std::regex dateRegex(
23            R"((\d{4})-(\d{2})-(\d{2}))");
24        std::smatch match;
25
26        if (!std::regex_match(val, match, dateRegex)) {
27            return false;
28        }
29
30        int y = std::stoi(match[1]);
31        int m = std::stoi(match[2]);
32        int d = std::stoi(match[3]);
33
34        return isValidDate(y, m, d);
35    }
36
37    std::string toString() const override {
38        char buffer[11];
39        sprintf(buffer, "%04d-%02d-%02d", year, month, day);
40        return std::string(buffer);
41    }
42
43    nlohmann::json toJSON() const override {
44        return nlohmann::json{
45            {"type", "DATE"},
46            {"year", year},
47            {"month", month},
48            {"day", day}
49        };
50    }
51
52 private:
53     bool isValidDate(int y, int m, int d) const {
54         if (m < 1 || m > 12) return false;
55         if (d < 1 || d > 31) return false;
56     }

```

```

57     int daysInMonth[] = {31,28,31,30,31,30,
58                          31,31,30,31,30,31};
59
60     if (m == 2 && isLeapYear(y)) {
61         return d <= 29;
62     }
63
64     return d <= daysInMonth[m-1];
65 }
66
67 bool isLeapYear(int y) const {
68     return (y % 4 == 0 && y % 100 != 0) ||
69           (y % 400 == 0);
70 }
71
72 bool isValid() const {
73     return isValidDate(year, month, day);
74 }
75 };

```

Listing 6: DateType - Date Type Implementation

4.5 Column Class

```

1 class Column {
2 private:
3     std::string name;
4     std::shared_ptr<DataType> dataType;
5     bool isPrimaryKey;
6     bool isNotNull;
7
8 public:
9     Column(const std::string& colName,
10            std::shared_ptr<DataType> type,
11            bool pk = false, bool notNull = false)
12         : name(colName), dataType(type),
13           isPrimaryKey(pk), isNotNull(notNull) {}
14
15     std::string getName() const { return name; }
16     std::shared_ptr<DataType> getType() const {
17         return dataType;
18     }
19     bool getIsPrimaryKey() const { return isPrimaryKey; }
20     bool getIsNotNull() const { return isNotNull; }
21
22     bool validateValue(const std::string& value) const {
23         if (isNotNull && value.empty()) {
24             return false;
25         }
26         return dataType->validate(value);
27     }
28
29     nlohmann::json toJSON() const {
30         return nlohmann::json{
31             {"name", name},
32             {"type", dataType->getTypeName()},
33             {"primaryKey", isPrimaryKey},

```

```

34         {"notNull", isNotNull}
35     };
36 }
37 };

```

Listing 7: Column - Column Definition

4.6 Row Class

```

1  class Row {
2  private:
3      std::vector<std::shared_ptr<DataType>> values;
4      int rowId;
5
6  public:
7      Row(int id = -1) : rowId(id) {}
8
9      void addValue(std::shared_ptr<DataType> value) {
10         values.push_back(value);
11     }
12
13     std::shared_ptr<DataType> getValue(size_t index) const {
14         if (index >= values.size()) {
15             throw std::out_of_range(
16                 "Column index out of range");
17         }
18         return values[index];
19     }
20
21     void setValue(size_t index,
22                  std::shared_ptr<DataType> value) {
23         if (index >= values.size()) {
24             throw std::out_of_range(
25                 "Column index out of range");
26         }
27         values[index] = value;
28     }
29
30     size_t getColumnCount() const {
31         return values.size();
32     }
33
34     int getRowId() const { return rowId; }
35     void setRowId(int id) { rowId = id; }
36
37     nlohmann::json toJSON() const {
38         nlohmann::json j;
39         j["rowId"] = rowId;
40         nlohmann::json valuesJson =
41             nlohmann::json::array();
42
43         for (const auto& val : values) {
44             valuesJson.push_back(val->toJSON());
45         }
46         j["values"] = valuesJson;
47         return j;
48     }

```

49 };

Listing 8: Row - Data Row

4.7 Table Class

```

1  class Table {
2  private:
3      std::string tableName;
4      std::vector<Column> columns;
5      std::vector<Row> rows;
6      int nextRowId;
7      std::mutex tableMutex;
8
9  public:
10     Table(const std::string& name)
11         : tableName(name), nextRowId(1) {}
12
13     void addColumn(const Column& column) {
14         std::lock_guard<std::mutex> lock(tableMutex);
15         columns.push_back(column);
16     }
17
18     bool insertRow(
19         const std::vector<std::string>& values) {
20         std::lock_guard<std::mutex> lock(tableMutex);
21
22         if (values.size() != columns.size()) {
23             return false;
24         }
25
26         Row newRow(nextRowId++);
27
28         for (size_t i = 0; i < columns.size(); ++i) {
29             if (!columns[i].validateValue(values[i])) {
30                 return false;
31             }
32
33             auto dataType = DataTypeFactory::createType(
34                 columns[i].getType()->getTypeName());
35             dataType->fromJson(
36                 nlohmann::json::parse(values[i]));
37             newRow.addValue(dataType);
38         }
39
40         rows.push_back(newRow);
41         return true;
42     }
43
44     bool updateRow(int rowId,
45                   const std::string& columnName,
46                   const std::string& newValue) {
47         std::lock_guard<std::mutex> lock(tableMutex);
48
49         int columnIndex = -1;
50         for (size_t i = 0; i < columns.size(); ++i) {
51             if (columns[i].getName() == columnName) {

```

```

52         columnIndex = i;
53         break;
54     }
55 }
56
57 if (columnIndex == -1) return false;
58
59 for (auto& row : rows) {
60     if (row.getRowId() == rowId) {
61         if (!columns[columnIndex]
62             .validateValue(newValue)) {
63             return false;
64         }
65
66         auto dataType =
67             DataTypeFactory::createType(
68                 columns[columnIndex].getType()
69                 ->getTypeName());
70         dataType->fromJSON(
71             nlohmann::json::parse(newValue));
72         row.setValue(columnIndex, dataType);
73         return true;
74     }
75 }
76
77 return false;
78 }
79
80 bool deleteRow(int rowId) {
81     std::lock_guard<std::mutex> lock(tableMutex);
82
83     auto it = std::remove_if(
84         rows.begin(), rows.end(),
85         [rowId](const Row& row) {
86             return row.getRowId() == rowId;
87         });
88
89     if (it != rows.end()) {
90         rows.erase(it, rows.end());
91         return true;
92     }
93
94     return false;
95 }
96
97 std::vector<Row> selectAll() const {
98     std::lock_guard<std::mutex> lock(
99         const_cast<std::mutex&>(tableMutex));
100     return rows;
101 }
102
103 std::vector<Column> getColumns() const {
104     return columns;
105 }
106
107 std::string getName() const {
108     return tableName;
109 }

```

110 };

Listing 9: Table - Table with CRUD Operations

4.8 Database Class

```

1  class Database {
2  private:
3      std::string dbName;
4      std::map<std::string,
5          std::shared_ptr<Table>> tables;
6
7  public:
8      Database(const std::string& name = "MainDB")
9          : dbName(name) {}
10
11     bool createTable(
12         const std::string& tableName,
13         const std::vector<Column>& columns) {
14         if (tableExists(tableName)) {
15             return false;
16         }
17
18         auto table =
19             std::make_shared<Table>(tableName);
20         for (const auto& col : columns) {
21             table->addColumn(col);
22         }
23
24         tables[tableName] = table;
25         return true;
26     }
27
28     bool dropTable(const std::string& tableName) {
29         auto it = tables.find(tableName);
30         if (it == tables.end()) {
31             return false;
32         }
33
34         tables.erase(it);
35         return true;
36     }
37
38     bool tableExists(
39         const std::string& tableName) const {
40         return tables.find(tableName) != tables.end();
41     }
42
43     std::shared_ptr<Table> getTable(
44         const std::string& tableName) {
45         auto it = tables.find(tableName);
46         if (it == tables.end()) {
47             return nullptr;
48         }
49         return it->second;
50     }
51 }

```

```
52     std::vector<std::string> getTableNames() const {
53         std::vector<std::string> names;
54         for (const auto& pair : tables) {
55             names.push_back(pair.first);
56         }
57         return names;
58     }
59
60     std::string getName() const { return dbName; }
61
62     bool saveToFile(
63         const std::string& filename) const {
64         nlohmann::json j;
65         j["dbName"] = dbName;
66
67         nlohmann::json tablesJson =
68             nlohmann::json::object();
69         for (const auto& pair : tables) {
70             tablesJson[pair.first] =
71                 pair.second->toJSON();
72         }
73         j["tables"] = tablesJson;
74
75         std::ofstream file(filename);
76         if (!file.is_open()) {
77             return false;
78         }
79
80         file << j.dump(4);
81         file.close();
82         return true;
83     }
84
85     bool loadFromFile(
86         const std::string& filename) {
87         std::ifstream file(filename);
88         if (!file.is_open()) {
89             return false;
90         }
91
92         nlohmann::json j;
93         file >> j;
94         file.close();
95
96         dbName = j["dbName"].get<std::string>();
97         tables.clear();
98
99         for (auto& [tableName, tableJson] :
100             j["tables"].items()) {
101             auto table =
102                 std::make_shared<Table>(tableName);
103             table->fromJSON(tableJson);
104             tables[tableName] = table;
105         }
106
107         return true;
108     }
```

```
109 };
```

Listing 10: Database - Collection of Tables

5 Client-Server Communication

5.1 JSON Protocol

Communication between client and server is done through standardized JSON messages.

5.1.1 Client to Server Message Format

CREATE TABLE Example:

```
1 {
2   "operation": "CREATE_TABLE",
3   "tableName": "Users",
4   "columns": [
5     {
6       "name": "id",
7       "type": "INT",
8       "primaryKey": true,
9       "notNull": true
10    },
11    {
12      "name": "username",
13      "type": "VARCHAR(50)",
14      "primaryKey": false,
15      "notNull": true
16    },
17    {
18      "name": "birthdate",
19      "type": "DATE",
20      "primaryKey": false,
21      "notNull": false
22    }
23  ]
24 }
```

INSERT Example:

```
1 {
2   "operation": "INSERT",
3   "tableName": "Users",
4   "values": [
5     {"type": "INT", "value": 1},
6     {"type": "VARCHAR(50)", "value": "gabriel"},
7     {"type": "DATE", "year": 2003,
8       "month": 5, "day": 15}
9   ]
10 }
```

UPDATE Example:

```
1 {
2   "operation": "UPDATE",
3   "tableName": "Users",
4   "rowId": 1,
```

```
5     "columnName": "username",
6     "newValue": {
7         "type": "VARCHAR(50)",
8         "value": "gabi_new"
9     }
10 }
```

DELETE Example:

```
1 {
2     "operation": "DELETE",
3     "tableName": "Users",
4     "rowId": 1
5 }
```

5.1.2 Server to Client Response Format

Success Response:

```
1 {
2     "status": "success",
3     "message": "Operation completed successfully",
4     "data": {}
5 }
```

Error Response:

```
1 {
2     "status": "error",
3     "message": "Table not found",
4     "errorCode": 404
5 }
```

5.2 Server Class

```
1 class Server {
2 private:
3     int serverSocket;
4     int port;
5     bool isRunning;
6     std::vector<std::thread> clientThreads;
7     DatabaseManager* dbManager;
8
9 public:
10     Server(int serverPort)
11         : port(serverPort), isRunning(false) {
12         dbManager = DatabaseManager::getInstance();
13     }
14
15     bool start() {
16         serverSocket = socket(
17             AF_INET, SOCK_STREAM, 0);
18         if (serverSocket < 0) {
19             return false;
20         }
21
22         int opt = 1;
23         setsockopt(serverSocket, SOL_SOCKET,
```

```
24         SO_REUSEADDR, &opt, sizeof(opt));
25
26     sockaddr_in serverAddr;
27     serverAddr.sin_family = AF_INET;
28     serverAddr.sin_addr.s_addr = INADDR_ANY;
29     serverAddr.sin_port = htons(port);
30
31     if (bind(serverSocket,
32             (struct sockaddr*)&serverAddr,
33             sizeof(serverAddr)) < 0) {
34         close(serverSocket);
35         return false;
36     }
37
38     if (listen(serverSocket, 5) < 0) {
39         close(serverSocket);
40         return false;
41     }
42
43     std::cout << "Server started on port "
44               << port << std::endl;
45     isRunning = true;
46
47     acceptClients();
48     return true;
49 }
50
51 void stop() {
52     isRunning = false;
53     close(serverSocket);
54
55     for (auto& thread : clientThreads) {
56         if (thread.joinable()) {
57             thread.join();
58         }
59     }
60 }
61
62 private:
63     void acceptClients() {
64         while (isRunning) {
65             sockaddr_in clientAddr;
66             socklen_t clientAddrLen =
67                 sizeof(clientAddr);
68
69             int clientSocket = accept(
70                 serverSocket,
71                 (struct sockaddr*)&clientAddr,
72                 &clientAddrLen);
73
74             if (clientSocket < 0) {
75                 if (isRunning) {
76                     std::cerr << "Accept failed"
77                               << std::endl;
78                 }
79                 continue;
80             }
81         }
```

```
82         clientThreads.emplace_back(  
83             &Server::handleClient,  
84             this, clientSocket);  
85     }  
86 }  
87  
88 void handleClient(int clientSocket) {  
89     ClientHandler handler(clientSocket, dbManager);  
90     handler.processRequests();  
91 }  
92 };
```

Listing 11: Server - Socket Server Implementation

5.3 ClientHandler Class

```
1 class ClientHandler {  
2 private:  
3     int clientSocket;  
4     DatabaseManager* dbManager;  
5  
6 public:  
7     ClientHandler(int socket, DatabaseManager* manager)  
8         : clientSocket(socket),  
9           dbManager(manager) {}  
10  
11     void processRequests() {  
12         char buffer[4096];  
13  
14         while (true) {  
15             memset(buffer, 0, sizeof(buffer));  
16             int bytesReceived = recv(  
17                 clientSocket, buffer,  
18                 sizeof(buffer) - 1, 0);  
19  
20             if (bytesReceived <= 0) {  
21                 break;  
22             }  
23  
24             buffer[bytesReceived] = '\0';  
25             std::string request(buffer);  
26  
27             nlohmann::json requestJson;  
28             try {  
29                 requestJson =  
30                     nlohmann::json::parse(request);  
31             } catch (const std::exception& e) {  
32                 sendError("Invalid JSON format");  
33                 continue;  
34             }  
35  
36             std::string operation =  
37                 requestJson["operation"]  
38                     .get<std::string>();  
39  
40             nlohmann::json response;  
41
```

```

42         if (operation == "CREATE_TABLE") {
43             response =
44                 handleCreateTable(requestJson);
45         } else if (operation == "INSERT") {
46             response = handleInsert(requestJson);
47         } else if (operation == "UPDATE") {
48             response = handleUpdate(requestJson);
49         } else if (operation == "DELETE") {
50             response = handleDelete(requestJson);
51         } else if (operation == "SELECT") {
52             response = handleSelect(requestJson);
53         } else {
54             response = createErrorResponse(
55                 "Unknown operation");
56         }
57
58         std::string responseStr = response.dump();
59         send(clientSocket, responseStr.c_str(),
60             responseStr.length(), 0);
61     }
62
63     close(clientSocket);
64 }
65
66 private:
67     nlohmann::json createSuccessResponse(
68         const std::string& message) {
69         return nlohmann::json{
70             {"status", "success"},
71             {"message", message}
72         };
73     }
74
75     nlohmann::json createErrorResponse(
76         const std::string& message) {
77         return nlohmann::json{
78             {"status", "error"},
79             {"message", message}
80         };
81     }
82 };

```

Listing 12: ClientHandler - Process Client Requests

5.4 DatabaseManager Class

```

1 class DatabaseManager {
2 private:
3     static DatabaseManager* instance;
4     Database database;
5     std::mutex dbMutex;
6     std::string dbFilePath;
7
8     DatabaseManager()
9         : database("MainDB"),
10           dbFilePath("mainDB.db") {
11         loadDatabase();

```

```

12     }
13
14 public:
15     static DatabaseManager* getInstance() {
16         if (instance == nullptr) {
17             instance = new DatabaseManager();
18         }
19         return instance;
20     }
21
22     bool createTable(
23         const std::string& tableName,
24         const std::vector<Column>& columns) {
25         std::lock_guard<std::mutex> lock(dbMutex);
26         bool result = database.createTable(
27             tableName, columns);
28         if (result) {
29             saveDatabase();
30         }
31         return result;
32     }
33
34     bool insertRow(const std::string& tableName,
35                   const std::vector<std::string>&
36                     values) {
37         std::lock_guard<std::mutex> lock(dbMutex);
38         auto table = database.getTable(tableName);
39         if (!table) return false;
40
41         bool result = table->insertRow(values);
42         if (result) {
43             saveDatabase();
44         }
45         return result;
46     }
47
48     void saveDatabase() {
49         database.saveToFile(dbFilePath);
50     }
51
52     void loadDatabase() {
53         if (!database.loadFromFile(dbFilePath)) {
54             std::cout << "Starting with new database"
55                       << std::endl;
56         }
57     }
58 };
59
60 DatabaseManager* DatabaseManager::instance = nullptr;

```

Listing 13: DatabaseManager - Business Logic

6 Client Implementation

```

1 class Client {
2 private:

```

```
3     int clientSocket;
4     std::string serverIP;
5     int serverPort;
6     bool isConnected;
7
8 public:
9     Client(const std::string& ip, int port)
10         : serverIP(ip), serverPort(port),
11           isConnected(false) {}
12
13     bool connect() {
14         clientSocket = socket(
15             AF_INET, SOCK_STREAM, 0);
16         if (clientSocket < 0) {
17             return false;
18         }
19
20         sockaddr_in serverAddr;
21         serverAddr.sin_family = AF_INET;
22         serverAddr.sin_port = htons(serverPort);
23
24         if (inet_pton(AF_INET, serverIP.c_str(),
25                     &serverAddr.sin_addr) <= 0) {
26             close(clientSocket);
27             return false;
28         }
29
30         if (::connect(clientSocket,
31                     (struct sockaddr*)&serverAddr,
32                     sizeof(serverAddr)) < 0) {
33             close(clientSocket);
34             return false;
35         }
36
37         isConnected = true;
38         return true;
39     }
40
41     void disconnect() {
42         if (isConnected) {
43             close(clientSocket);
44             isConnected = false;
45         }
46     }
47
48     nlohmann::json sendRequest(
49         const nlohmann::json& request) {
50         if (!isConnected) {
51             return nlohmann::json{
52                 {"status", "error"},
53                 {"message", "Not connected"}
54             };
55         }
56
57         std::string requestStr = request.dump();
58         send(clientSocket, requestStr.c_str(),
59             requestStr.length(), 0);
60     }
```

```

61     char buffer[4096];
62     memset(buffer, 0, sizeof(buffer));
63     int bytesReceived = recv(clientSocket,
64                             buffer,
65                             sizeof(buffer) - 1,
66                             0);
67
68     if (bytesReceived <= 0) {
69         return nlohmann::json{
70             {"status", "error"},
71             {"message", "Connection lost"}
72         };
73     }
74
75     buffer[bytesReceived] = '\0';
76
77     try {
78         return nlohmann::json::parse(buffer);
79     } catch (const std::exception& e) {
80         return nlohmann::json{
81             {"status", "error"},
82             {"message", "Invalid response"}
83         };
84     }
85 }
86 };

```

Listing 14: Client Application

7 Build System - CMake

```

1 cmake_minimum_required(VERSION 3.10)
2 project(DatabaseSystem VERSION 1.0)
3
4 set(CMAKE_CXX_STANDARD 14)
5 set(CMAKE_CXX_STANDARD_REQUIRED True)
6 set(CMAKE_CXX_FLAGS "-Wall -Wextra")
7
8 include_directories(${PROJECT_SOURCE_DIR}/include)
9
10 find_package(nlohmann_json REQUIRED)
11 find_package(Threads REQUIRED)
12
13 set(SOURCES
14     src/types/IntType.cpp
15     src/types/VarcharType.cpp
16     src/types/DateType.cpp
17     src/database/Column.cpp
18     src/database/Row.cpp
19     src/database/Table.cpp
20     src/database/Database.cpp
21     src/manager/DatabaseManager.cpp
22     src/server/Server.cpp
23     src/server/ClientHandler.cpp
24 )
25

```

```
26 add_executable(db_server
27     ${SOURCES}
28     src/main_server.cpp
29 )
30
31 target_link_libraries(db_server
32     nlohmann_json::nlohmann_json
33     Threads::Threads
34 )
35
36 add_executable(db_client
37     src/client/Client.cpp
38     src/main_client.cpp
39 )
40
41 target_link_libraries(db_client
42     nlohmann_json::nlohmann_json
43 )
```

Listing 15: CMakeLists.txt

7.1 Build and Run

```
1 # Create build directory
2 mkdir build
3 cd build
4
5 # Configure
6 cmake ..
7
8 # Build
9 make
10
11 # Run server
12 ./db_server
13
14 # Run client (in separate terminal)
15 ./db_client
```

Listing 16: Build Commands

8 Advanced Features

8.1 Thread Safety

The system implements complete synchronization mechanisms for concurrent operations:

- **std::mutex** at table level - each table has its own mutex
- **std::lock_guard** for automatic locking (RAII pattern)
- **DatabaseManager mutex** - protects database-level operations

8.2 Data Persistence

The database is automatically saved to the `mainDB.db` file after each modification operation (INSERT, UPDATE, DELETE, CREATE TABLE).

When the server starts, the database is automatically loaded from file, ensuring data continuity between sessions.

8.3 Error Handling

The system implements robust error handling:

- Data type validation
- Table existence verification
- JSON parsing exception handling
- Connection timeouts
- Descriptive error messages

9 Testing and Examples

9.1 Complete Usage Example

```
1 // 1. Create table Users
2 CREATE_TABLE: Users
3 Columns: id(INT, PK), username(VARCHAR), birthdate DATE)
4
5 // 2. Insert users
6 INSERT: Users
7 Values: 1, "gabriel", 2003-05-15
8
9 // 3. Update username
10 UPDATE: Users
11 RowId: 1, Column: username, New: "gabi_new"
12
13 // 4. Select all users
14 SELECT: Users
15
16 // 5. Delete user
17 DELETE: Users
18 RowId: 1
```

Listing 17: Complete Workflow Example

9.2 Test Scenarios

1. Concurrency Test

- Connect multiple clients simultaneously
- Concurrent INSERTs on same table

- Verify data integrity

2. Persistence Test

- Create data in session 1
- Restart server
- Verify data in session 2

3. Validation Test

- Attempt INSERT with wrong types
- VARCHAR exceeding character limit
- Invalid DATE

4. CRUD Operations Test

- CREATE TABLE with multiple columns
- INSERT multiple rows
- UPDATE partial
- DELETE selective
- SELECT with result verification

10 Possible Improvements

10.1 Future Functionalities

- **Query Language** - SQL-like implementation for complex commands
- **Indexing** - B-tree indexing for fast searches
- **Transactions** - ACID support with BEGIN, COMMIT, ROLLBACK
- **Constraints** - FOREIGN KEY, UNIQUE, CHECK constraints
- **JOIN Operations** - Support for joins between tables
- **Authentication** - User system with permissions
- **Backup/Restore** - Automatic backup mechanisms
- **Query Optimization** - Performance optimization
- **GUI Client** - Graphical interface for client
- **Logging** - Complete logging system for audit

10.2 Performance Optimizations

- Thread pool for efficient client management
- Connection pooling
- Caching layer for frequent queries
- Async I/O for file saving
- Data compression for network transfer

11 Conclusions

11.1 Accomplished Objectives

The project successfully achieved:

- Functional client-server architecture
- Support for multiple simultaneous connections
- Complete CRUD operations
- Data persistence
- Thread-safety for concurrent operations
- Standardized communication protocol (JSON)
- Modular and extensible OOP design

11.2 Lessons Learned

Developing this system provided valuable experience in:

- Design patterns (Singleton, Factory, Command)
- Multi-threaded programming and synchronization
- Socket programming and network protocols
- JSON serialization/deserialization
- Layered architectures and separation of responsibilities
- SOLID principles in practice
- Build systems (CMake)

11.3 Applicability

The knowledge and techniques used in this project are directly applicable in:

- Distributed systems development
- Backend development
- Data management systems
- Client-server applications
- Microservices architecture

Appendices

A. Complete Project Structure

```
DatabaseManagementSystem/  
|-- CMakeLists.txt  
|-- include/  
|   |-- types/  
|   |   |-- DataType.h  
|   |   |-- IntType.h  
|   |   |-- VarcharType.h  
|   |   |-- DateType.h  
|   |   +--- DataTypeFactory.h  
|   |-- database/  
|   |   |-- Column.h  
|   |   |-- Row.h  
|   |   |-- Table.h  
|   |   +--- Database.h  
|   |-- server/  
|   |   |-- Server.h  
|   |   +--- ClientHandler.h  
|   |-- manager/  
|   |   +--- DatabaseManager.h  
|   +--- client/  
|       +--- Client.h  
|-- src/  
|   |-- types/  
|   |-- database/  
|   |-- server/  
|   |-- manager/  
|   |-- client/  
|   |-- main_server.cpp  
|   +--- main_client.cpp  
|-- build/  
|   |-- db_server  
|   +--- db_client  
+--- mainDB.db
```

B. References and Resources

- C++ Reference: <https://en.cppreference.com>
- nlohmann/json: <https://github.com/nlohmann/json>
- CMake Documentation: <https://cmake.org/documentation/>
- POSIX Sockets: <https://man7.org/linux/man-pages/>
- Design Patterns: Gang of Four